

Frontend, Backend & the Server

how a full web app fits together



Dinesh Rawat

Instructor · Shorter Loop × Snapied

1 The two halves of every web app

Every website you use is split into two sides that talk to each other:

- **Frontend** - what you see and click, running in your **browser**
- **Backend** - the logic and data, running on a **server** somewhere else

The frontend asks, the backend answers. You have now built pieces of both.

2 Frontend - what the user sees

The frontend builds the **user interface**. It runs inside the browser and handles what the user sees and does.

languages

- **HTML** - structure
- **CSS** - styling
- **JavaScript** - behaviour

frameworks & libraries

- **React, Angular, Vue.js** - the modern big three
- **jQuery, Backbone.js** - older tools you will still see around

Frontend does: the UI, user interactions, animations, forms, and rendering the page. It all runs in the browser.

3 Backend - the engine behind it

The backend runs on the **server**, not in the browser. The user never sees it directly. It does the real work behind the scenes.

languages & frameworks

- **Node.js + Express.js** - JavaScript on the server (what we are using)
- **Python, Java, PHP, ASP.NET, JSP** - other backend choices

what the backend does

- runs the **business logic** (the rules of the app)
- connects to the **database**
- handles **authentication & authorization** (who are you, what are you allowed to do)
- builds the **APIs** the frontend calls
- sends **data back** to the frontend

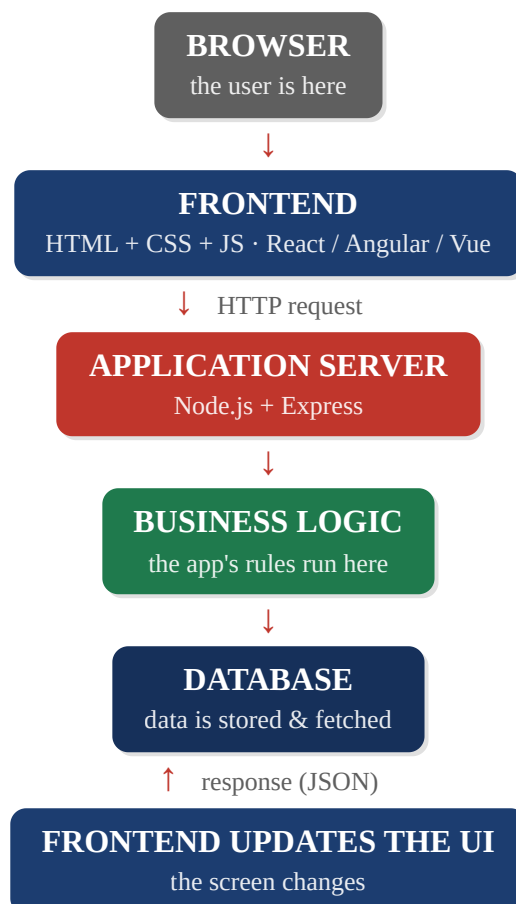
4 The application server

The **application server** is the software that sits in the middle. It receives requests, runs the backend logic, and sends responses back.

For us, that is Node.js + Express. JavaScript runs on the server through Node, and Express is the framework that handles the requests. Together they are our application server, the thing that takes a client request and returns a response.

5 The complete web flow

Here is the whole journey, from a click in the browser to the screen updating:



Read it as a round trip: the request travels down (browser to database), then the response (usually JSON) travels back up, and the frontend redraws the screen with the new data.

6 Frontend vs Backend at a glance

	Frontend	Backend
Runs in	the browser	the server
User sees it?	yes, directly	no, hidden
Job	UI, interaction, display	logic, data, security
Languages	HTML, CSS, JS	Node, Python, Java, PHP
Frameworks	React, Angular, Vue	Express, Django, Spring
Talks to	the user + the API	the API + the database

7 Say it in one line (for interviews)

frontend

What users see and interact with.

backend

Processes requests, runs business logic, and handles the database.

application server

Software (like Node.js + Express) that receives HTTP requests, runs the backend logic, and sends responses back to the client.

The whole thing in one breath: the frontend asks, the application server processes and talks to the database, then sends back JSON, and the frontend updates the screen.

Full-Stack Internship · Module 4 · Backend

Day 13 · Task

Task: Build the API Routes

the full REST API for our AI Resume Builder



Dinesh Rawat

Instructor · Shorter Loop × Snapied

Today you learned Express. Now build the backend for the product we keep coming back to: the **AI Resume Builder**. Your job is to create **every route for every resource** listed here.

the rules

- Use **Express**, one server, clean route files

- Store data in a **JSON file** (read it on start, write it on change). No database yet, that is next module.
- Turn on `express.json()` so `req.body` works
- Every route returns **JSON** and the right **status code** (200, 201, 204, 400, 401, 404)
- You do not need real login or real AI yet. For auth and AI routes, return a sensible **fake/mock response**, the point today is the **routes and shapes**, not the logic behind them
- Use the right **method** for each job (GET read, POST create, PUT replace, PATCH update, DELETE remove)

Goal: a running API where every endpoint below responds. Test each one in the browser or with a REST client. If it returns JSON with the correct status, that route is done.

1 Auth

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Obtain an access token
POST	/api/auth/logout	Invalidate the session
POST	/api/auth/forgot-password	Begin password recovery
POST	/api/auth/reset-password	Complete password reset

2 Users

Method	Endpoint	Purpose
GET	/api/users/me	Current profile, tier, AI credits
PUT	/api/users/me	Update profile
DELETE	/api/users/me	Delete account and data

3 Documents (the core resource)

Method	Endpoint	Purpose
GET	/api/documents	List my resumes and cover letters
POST	/api/documents	Create one (blank or from a template)
POST	/api/documents/import	Create one from an upload or LinkedIn data
GET	/api/documents/:id	Read one with its full content
PUT	/api/documents/:id	Save edits
POST	/api/documents/:id/duplicate	Copy it (a tailored version)
DELETE	/api/documents/:id	Delete it

4 Sections & items (nested under a document)

Method	Endpoint	Purpose
POST	/api/documents/:id/sections	Add a section
PATCH	/api/documents/:id/sections/:sectionId	Edit or reorder a section
DELETE	/api/documents/:id/sections/:sectionId	Remove a section
POST	/api/.../sections/:sectionId/items	Add an entry
PATCH	/api/.../items/:itemId	Edit or reorder an entry
DELETE	/api/.../items/:itemId	Remove an entry

Note: many builders just save the whole document in one [PUT /api/documents/:id](#) instead of editing each section over the wire. Both are valid. The nested routes matter more if you want autosave-per-field. Build the nested routes anyway, so you have them.

5 Versions

Method	Endpoint	Purpose
GET	/api/documents/:id/versions	List saved versions
POST	/api/documents/:id/versions	Save the current state as a version
POST	/api/documents/:id/versions/:versionId/restore	Roll back to one

6 Templates

Method	Endpoint	Purpose
GET	/api/templates	List available designs
GET	/api/templates/:id	One template's config

7 AI (action resource, each call is metered)

Method	Endpoint	Purpose
POST	/api/ai/bullets	Generate or improve bullet points
POST	/api/ai/summary	Generate a summary or headline
POST	/api/ai/rewrite	Tighten or improve selected text
POST	/api/ai/prompt	Apply a freeform instruction to a section

For the AI routes: you do not need a real AI. Just accept the input and return a mock response (for example, send back the text with " (improved)" added). The routes and JSON shapes are what matter today.

8 Applications (job tracker)

Method	Endpoint	Purpose
GET	/api/applications	List tracked applications
POST	/api/applications	Log one
PATCH	/api/applications/:id	Update status
DELETE	/api/applications/:id	Remove one

Also mentioned in the full spec (build these if you have time): ATS check, tailoring, exports, and sharing. Same pattern, POST for an action, GET to read. Get the eight resources above working first.

u26 How to approach it

- Start with **Documents**, it is the core. Get full CRUD working first (list, create, read, update, delete).
- Store everything in a **data.json** file. Read it into memory on start, write back on every change.
- Give each new item an **id** so **:id** routes can find it.

- Do the other resources one at a time. Auth and AI can return mock data.
- Test every route as you go. Do not write all of them then debug at the end.

```
// suggested structure
resume-api/
  app.js
  routes/
    auth.js  users.js  documents.js
    templates.js  ai.js  applications.js
  data.json
  README.md
```

submit

- Push to a repo named **resume-api**, from the terminal
- In the README, list every route you built and how to run the server
- Share the repo link

Done means: the server runs, and every endpoint in this document responds with JSON and the correct status code. That is a real backend for a real product.